

Sri Lanka Institute of Information Technology

Faculty of Computing

IT2130 - Operating Systems and System Administration

Dr.Sanvitha Kasthuriarachchi

Year 02 and Semester 02

Lecture 05

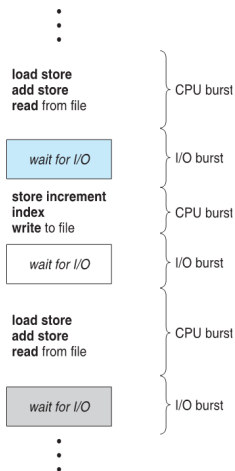
CPU Scheduling in Operating Systems

CPU Scheduling

- CPU scheduling Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Scheduling Algorithm Evaluation

Basic Concepts

- Multiple programs reside in the main memory to maximum CPU utilization.
- The PCBs of the processes are loaded in the ready queue in the memory.
- Every process has CPU burst periods and I/O Burst periods. It consists of **CPU bursts** followed by **I/O bursts**.
- CPU burst distribution is the main concern in CPU scheduling.



Preemptive and Non-preemptive Scheduling

- **Preemptive**: when a process is in the CPU, the execution can be suspended at any point.
- **Non-preemptive**: when a process is in the CPU, the execution cannot be suspended until completes the entire execution.

CPU Scheduling

- **Short-term scheduler/ CPU scheduler** selects the processes (PCBs) in the ready queue and allocates the CPU.
- CPU scheduling decisions may take place when a process:
 - Switches from running to waiting state
 - Switches from running to ready state
 - Switches from waiting to ready
 - Terminates

Dispatcher

- The **dispatcher** is a module that transfers control of the CPU from one process to another.
- The key functions are;
 - Perform the context switch
 - Switch to user mode
 - Restart the new user program.
- **Dispatch latency** is the time takes for the dispatcher to stop one process and start another to run.

Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – number of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced.

Optimization Criteria for Scheduling Algorithm

- Maximize the CPU utilization
- Maximize the throughput
- Minimize the turnaround time
- Minimize the waiting time
- Minimize the response time

First- Come, First-Served (FCFS) Scheduling

Suppose that the processes arrive at the same time in the order: P_1, P_2, P_3

Process	Burst Time (ms)
P1	24
P2	3
P3	3

The Gantt Chart for the schedule is:



Waiting time for processes;

- $P_1 = 0$ ms
- $P_2 = 24$ ms
- $P_3 = 27$ ms

Average waiting time: $(0 + 24 + 27)/3$
 $= 17$ ms

Turnaround time for processes;

- $P_1 = 24$ ms
- $P_2 = 27$ ms
- $P_3 = 30$ ms

Average Turnaround time: $(24 + 27 + 30)/3 = 27$ ms

First- Come, First-Served (FCFS) Scheduling

Suppose that the processes arrive at the same time in the order: P_2, P_3, P_1

Process	Burst Time (ms)
P1	24
P2	3
P3	3

The Gantt Chart for the schedule is:



Waiting time for processes;

- $P_1 = 6$ ms
- $P_2 = 0$ ms
- $P_3 = 3$ ms

Average waiting time: $(6 + 0 + 3)/3 = 3$ ms

Turnaround time for processes;

- $P_1 = 30$ ms
- $P_2 = 3$ ms
- $P_3 = 6$ ms

Average Turnaround time: $(30 + 3 + 6)/3 = 13$ ms

First- Come, First-Served (FCFS) Scheduling

- First- Come, First-Served scheduling is one of the simplest types of **Non- preemptive** CPU scheduling algorithms.
- The processes are attended to in the order in which they arrive in the ready queue.
- FCFS can result in long waiting times, if a long process arrives before a shorter one.
- This is known as the **Convoy effect**, where shorter processes are forced to wait behind longer processes, leading to inefficient execution.

Shortest-Job-First (SJF) Scheduling

- Shortest-Job-First is a scheduling process that selects the waiting process with the smallest execution time to execute next.
- This may or may not be preemptive.
- This is the optimal scheduling algorithm which gives minimum average waiting time for a given set of processes.
- However, this is not practically feasible, as it is difficult to know the length of the next/ future CPU request.

Shortest-Job-First (Non- Preemptive) Scheduling

Process	Burst Time (ms)
P1	6
P2	8
P3	7
P4	3

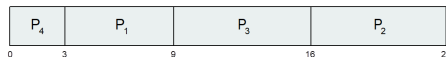
Waiting time for processes;

- $P1 = 3\text{ms}$
- $P2 = 16\text{ms}$
- $P3 = 9\text{ms}$
- $P4 = 0\text{ ms}$

Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7\text{ms}$

Suppose that the processes arrive at the same time.

The Gantt Chart for the schedule is:



Turnaround time for processes;

- $P1 = 9\text{ms}$
- $P2 = 24\text{ms}$
- $P3 = 16\text{ms}$
- $P4 = 3\text{ ms}$

Average Turnaround time = $(9 + 24 + 16 + 3) / 4 = 13\text{ms}$

Shortest-Job-First (Preemptive) Scheduling

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Waiting time for processes;

- $P1 = (0-0) + (10-1) = 9\text{ms}$
- $P2 = (1-1) = 0\text{ms}$
- $P3 = (17-2) = 15\text{ms}$
- $P4 = (5-3) = 2\text{ms}$

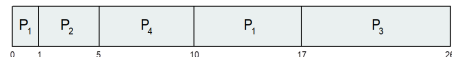
Average waiting time = $(9 + 0 + 15 + 2) / 4 = 6.5\text{ms}$

This is known as

Shortest-remaining-time-first scheduling.

Suppose that the processes arrive at different timeslots.

The Gantt Chart for the schedule is:



Turnaround time for processes;

- $P1 = 17\text{ms}$
- $P2 = (5-1) = 4\text{ms}$
- $P3 = (26 - 2) = 24\text{ms}$
- $P4 = (10-3) = 7\text{ms}$

Average Turnaround time = $(17+4+24+7) / 4 = 13\text{ms}$

Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
- This may be preemptive or non- preemptive.
- The low priority processes may never execute. This is known as **Starvation**
- As a solution to this problem, the priority increases with the time. It is known as **Aging**.

Priority Scheduling

Process	Burst Time (ms)	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

- Priority scheduling Gantt Chart



Waiting time for processes;

- P1 = 6ms
- P2 = 0ms
- P3 = 16ms
- P4 = 18ms
- P5 = 1ms

Average waiting time:
 $(6+0+16+18+1) / 5 = 8.2 \text{ ms}$

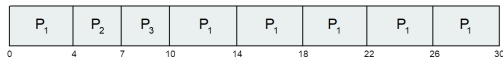
Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum q**), usually 10-100 ms. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- The Timer interrupts every quantum to schedule next process
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Assume there are 4 processes and the time quantum is 10 ms, then each process gets $1/4$ of the CPU and runs at most 10 ms per turn. The maximum waiting time before a process gets the CPU again $(4 - 1) \times 10 \text{ ms} = 30 \text{ ms}$

Round Robin (RR) with quantum (q) = 4

Process	Burst Time (ms)
P1	24
P2	3
P3	3

- The Gantt chart is:



Waiting time of processes;

- $P1 = (10 - 4) = 6\text{ms}$
- $P2 = 0\text{ms}$
- $P3 = 0\text{ms}$

Average waiting time: $(6+0+0) / 3 = 2\text{ms}$

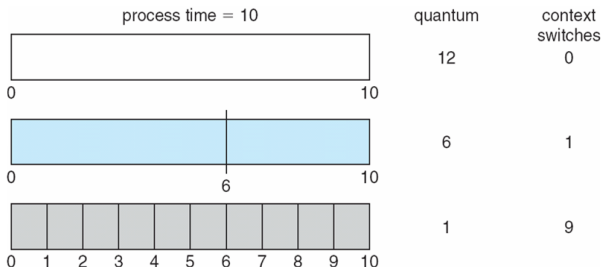
Turnaround time of processes

- $P1 = 30\text{ms}$
- $P2 = 7\text{ms}$
- $P3 = 10\text{ms}$

Average turnaround time = $(30+7+10)/3 = 15.67\text{ms}$

Time Quantum and Context Switch Time

- q should be large compared to context switch time
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high

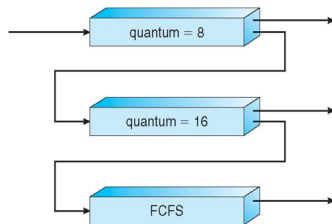


Multilevel Queue

- Ready queue is partitioned into separate queues
 - foreground
 - background
- Each queue has its own scheduling algorithm:
 - foreground – RR
 - background – FCFS
- Processes are permanently in a given queue and
- Scheduling must be done between the queues:
 - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR, 20% to background in FCFS

Multilevel Feedback Queue

- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service



CPU Scheduling Algorithm Evaluation

There are four evaluation methods.

- Deterministic evaluation
- Queueing Models
- Simulations
- Implementation

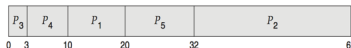
Deterministic Evaluation

- This is a type of **analytic evaluation**. It takes a particular predetermined workload and defines the performance of each algorithm for that workload.
- This is the simplest and fastest, but requires exact numbers for input, applies only to those inputs
- Consider 5 processes arriving at time 0: For each algorithm, calculate minimum average waiting time

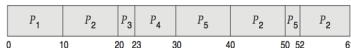
- FCS is 28ms:



- Non-preemptive SFJ is 13ms:



- RR is 23ms:



Process	Burst Time
P ₁	10
P ₂	29
P ₃	3
P ₄	7
P ₅	12

Queueing Models

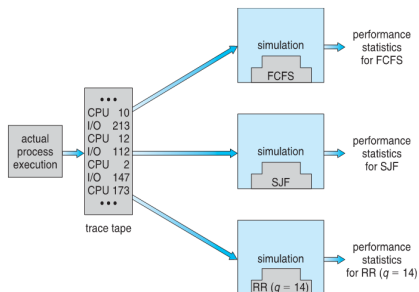
- This computation is done by Little's law.
Little's law – in steady state, processes leaving queue must equal processes arriving.

$n = \lambda \times W$ (n = average queue length, W = average waiting time in queue, λ = average arrival rate into queue)

- If on average 7 processes arrive per second, and normally 14 processes in queue, then average wait time per process = 2 seconds

Simulations

- Programmed model of computer system
- The clock is a variable. Random number generator according to probabilities, Trace tapes record sequences of real events in real systems
- It gathers statistics indicating algorithm performance
- **Simulation** is more accurate



Implementation

- Implement new scheduler and test in real systems.
- This requires high cost and a high-risk evaluation.
- Most flexible schedulers can be modified per-site or per-system or APIs to modify priorities
- The problem here is the scheduler must be changed according to the changes of the environment.

End of Lecture 5

Weekly plan

- **Practical:** Thread implementation in C
- **Workshop:**
 - Summary – CPU Scheduling in Operating Systems
 - Discussion of Tutorial 5

